

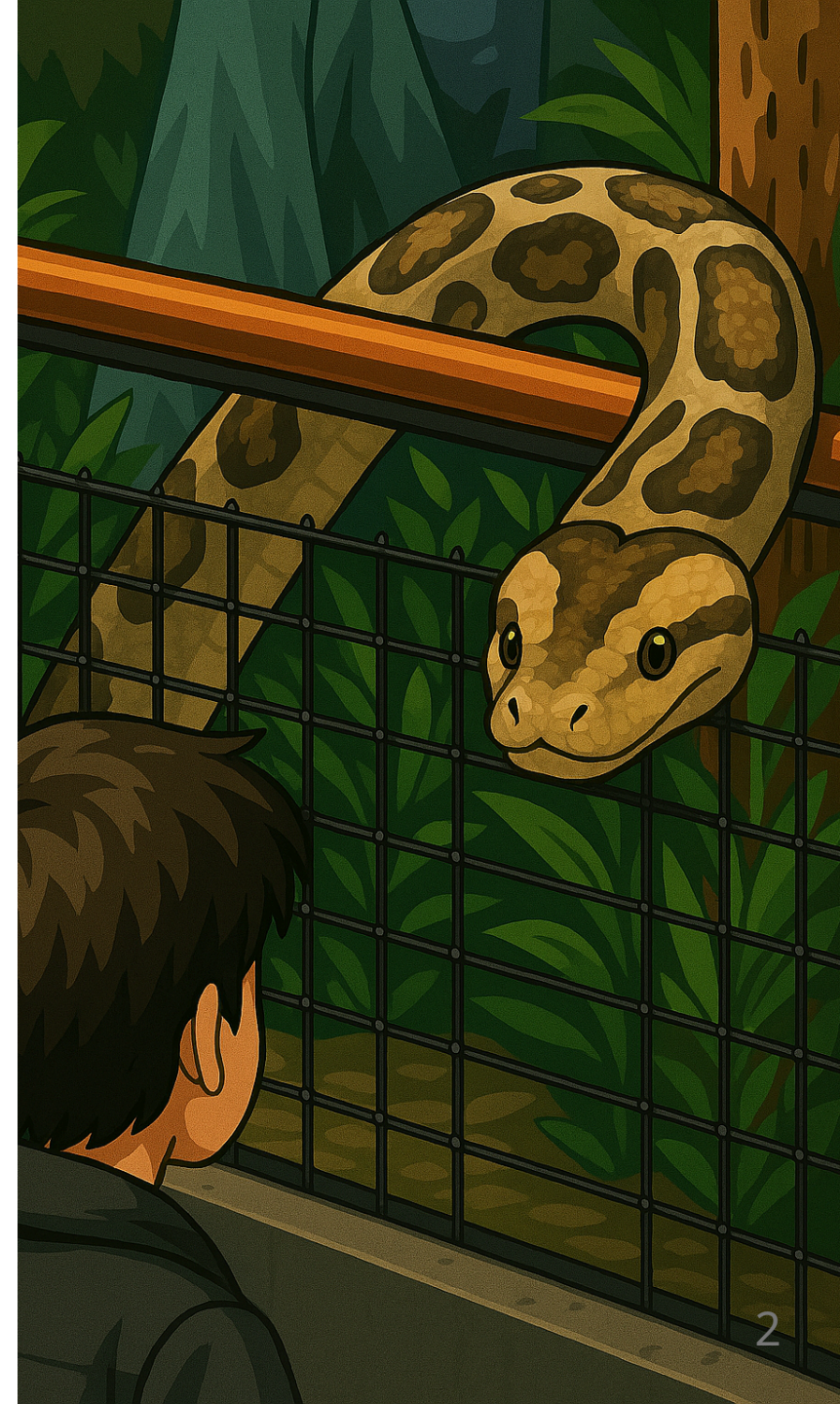
Python - structure de données

BTS CIEL



Sommaire

- Algorithmes & SDD
- Structure de données simples
 - Liste
 - N-uplet
 - Dictionnaire
 - Ensemble
- Parcours à l'aide d'une boucle
- Exercices



Algorithmes & structure de données

Les algorithmes et les structures de données sont les **deux piliers fondamentaux** de la programmation.

Ils ne peuvent pas être pensés séparément : **l'un influence directement l'efficacité de l'autre.**

Le choix de la structure de données détermine :

- la rapidité des traitements (temps d'accès, insertion, suppression),
- la mémoire utilisée,
- la lisibilité et la modularité du code.

 Un algorithme bien conçu peut devenir inefficace s'il repose sur une mauvaise structure de données.

Liste

Une liste permet de stocker une suite d'éléments de même type (ou non !)

```
couleurs = ["rouge", "vert", "bleue"]
notes = [10, 12, 12, 18]

# Contrairement aux tableaux du langage C la liste n'a pas de taille fixe
notes.append(20)
```

Les éléments sont accessibles via leur index

```
rouge = couleurs[0]
notes = notes[2]
derniere_note = notes[-1] # indexation négative

print(rouge)
# Résultat : "rouge"
print(derniere_note)
# Résultat : 20
```

Liste - opérations non destructives

Opération	Définition
<code>lst[i]</code>	Accède à l'élément d'indice <code>i</code>
<code>len(lst)</code>	Renvoie le nombre d'éléments
<code>x in lst</code>	Vérifie si <code>x</code> est présent dans la liste
<code>lst.index(x)</code>	Donne l'indice de la première occurrence de <code>x</code>
<code>lst.count(x)</code>	Compte le nombre d'occurrences de <code>x</code>
<code>lst.copy()</code>	Renvoie une copie indépendante de la liste
<code>sorted(lst)</code>	Renvoie une nouvelle liste triée
<code>lst1 + lst2</code>	Concatène deux listes
<code>lst * n</code>	Répète la liste <code>n</code> fois

i Une opération non destructives ne modifie pas la liste originale (ici `lst`).

Liste - opérations destructives

Opération	Définition
<code>lst[i] = x</code>	Remplace l'élément à l'indice <code>i</code> par <code>x</code>
<code>lst.append(x)</code>	Ajoute <code>x</code> à la fin de la liste
<code>lst.insert(i, x)</code>	Insère <code>x</code> à l'indice <code>i</code>
<code>lst.remove(x)</code>	Supprime la première occurrence de <code>x</code>
<code>lst.pop(i)</code>	Supprime et retourne l'élément d'indice <code>i</code>
<code>lst.sort()</code>	Trie la liste en place
<code>lst.reverse()</code>	Inverse l'ordre des éléments
<code>lst.clear()</code>	Supprime tous les éléments de la liste

N-uplet

Comme une liste, un n-uplet (tuple) permet de stocker une suite d'éléments

```
personne = (56, 35, "Jean Dupont")  
  
departement = personne[0]  
age = personne[1]
```

Les éléments sont aussi accessibles par destructuration

```
personne = (56, 35, "Jean Dupont")  
  
departement, age, nom = personne
```

N-uplet VS liste

Les tuples et les listes offrent des possibilités proches, mais leurs usages diffèrent :

- Les tuples sont **immuables** et servent souvent à stocker des éléments de types variés, accédés via leur position.
- Les listes sont **mutables**, contiennent en général des éléments de même type, généralement les éléments d'une liste sont traités en ensemble à l'aide d'une boucle.

 **Immuable** signifie qui ne peut pas être modifié après sa création

N-uplet - cas courant d'utilisation

Il est fréquent d'utiliser un N-uplet en résultat de fonction pour retourner plusieurs valeurs :

```
def swap(a, b):  
    return b, a  
  
a, b = 10, 2  
a, b = swap(a, b)  
  
print(a) # 2  
print(b) # 10
```

N-uplet - opérations non-destructives

Opération	Définition
<code>t[i]</code>	Accès à l'élément à l'indice <code>i</code>
<code>len(t)</code>	Renvoie le nombre d'éléments
<code>x in t</code>	Vérifie si <code>x</code> est présent
<code>t.count(x)</code>	Compte le nombre d'occurrences de <code>x</code>
<code>t.index(x)</code>	Donne l'indice de la première occurrence de <code>x</code>
<code>t1 + t2</code>	Concatène deux tuples
<code>t * n</code>	Répète le tuple <code>n</code> fois

Dictionnaire

Le dictionnaire est un ensemble de paires *clé-valeur* au sein duquel les clés doivent être uniques.

```
personne = {"nom": "Jean Dupont", "age": 35, "departement": 56, "couleur_preferree": "rouge"}
```

L'accès à une valeur se fait en utilisant la clé :

```
# Lecture
nom = personne["nom"]

# Ecriture
personne["couleur_preferree"] = "vert"

print(nom) # Résultat : "Jean Dupont"
print(personne["couleur_preferree"]) # Résultat : "vert"
```

Dictionnaire - opérations non-destructives

Opération	Description
<code>d[k]</code>	Accède à la valeur associée à la clé <code>k</code>
<code>k in d</code>	Vérifie si la clé <code>k</code> existe dans le dictionnaire
<code>len(d)</code>	Renvoie le nombre de paires clé/valeur
<code>d.get(k)</code>	Renvoie la valeur associée à <code>k</code> , ou <code>None</code> si absente
<code>d.keys()</code>	Renvoie une vue des clés
<code>d.values()</code>	Renvoie une vue des valeurs
<code>d.items()</code>	Renvoie une vue des paires <code>(clé, valeur)</code>
<code>dict(d)</code>	Crée une copie du dictionnaire

Dictionnaire - opérations destructives

Opération	Description
<code>d[k] = v</code>	Ajoute ou modifie une paire clé/valeur
<code>del d[k]</code>	Supprime la clé <code>k</code> et sa valeur associée
<code>d.pop(k)</code>	Supprime et retourne la valeur associée à <code>k</code>
<code>d.popitem()</code>	Supprime et retourne une paire clé/valeur (aléatoire avant Python 3.7, dernier après)
<code>d.clear()</code>	Vide complètement le dictionnaire
<code>d.update(other)</code>	Ajoute ou remplace des éléments à partir d'un autre dictionnaire
<code>d.setdefault(k, v)</code>	Ajoute la clé <code>k</code> avec valeur <code>v</code> si elle n'existe pas

Ensemble

Un ensemble (set) est une collection **non ordonnée** d'éléments sans doublons.

```
panier = {'pomme', 'orange', 'pomme', 'poire', 'orange', 'banane'}  
  
print(panier)  
# Résultat = {'orange', 'banane', 'poire', 'pomme'}
```

Ensemble - opérations non-destructives

Opération	Description
<code>x in s</code>	Vérifie si <code>x</code> est dans l'ensemble
<code>len(s)</code>	Renvoie le nombre d'éléments
<code>s.copy()</code>	Renvoie une copie de l'ensemble
<code>s.union(t)</code>	Nouvel ensemble contenant les éléments de <code>s</code> et <code>t</code>
<code>s.intersection(t)</code> ou <code>s & t</code>	Nouvel ensemble contenant les éléments communs
<code>s.difference(t)</code> ou <code>s - t</code>	Nouvel ensemble avec les éléments de <code>s</code> mais pas de <code>t</code>
<code>s.symmetric_difference(t)</code> ou <code>s ^ t</code>	Nouvel ensemble des éléments exclusifs à <code>s</code> ou <code>t</code>

Ensemble - opérations destructives

Opération	Description
<code>s.add(x)</code>	Ajoute <code>x</code> à l'ensemble
<code>s.remove(x)</code>	Supprime <code>x</code> (erreur si absent)
<code>s.discard(x)</code>	Supprime <code>x</code> (sans erreur si absent)
<code>s.pop()</code>	Supprime et retourne un élément arbitraire
<code>s.clear()</code>	Vide complètement l'ensemble
<code>s.update(t)</code>	Ajoute tous les éléments de l'itérable <code>t</code>

Comment choisir ?

Structure	Ordonnée	Mutable	Doublons autorisés	Accès par clé/indice	Types d'éléments	Remarques principales
Liste	✔ Oui	✔ Oui	✔ Oui	✔ Indice	Souvent homogènes	Structure de base, très flexible
Tuple	✔ Oui	✘ Non	✔ Oui	✔ Indice	Souvent hétérogènes	Léger, immuable, utilisable comme clé de dictionnaire
Ensemble	✘ Non	✔ Oui	✘ Non	✘ Pas d'indice	Homogènes conseillés	Pas de doublons, très rapide pour <code>in</code>
Dictionnaire	✘ Non	✔ Oui	✘ Non (clés)	✔ Clé	Valeurs libres	Association clé → valeur

Instruction `in`

L'instruction `in` est utilisable pour **chaque type de collection**. Elle permet de **vérifier** qu'une collection **contient un élément**.

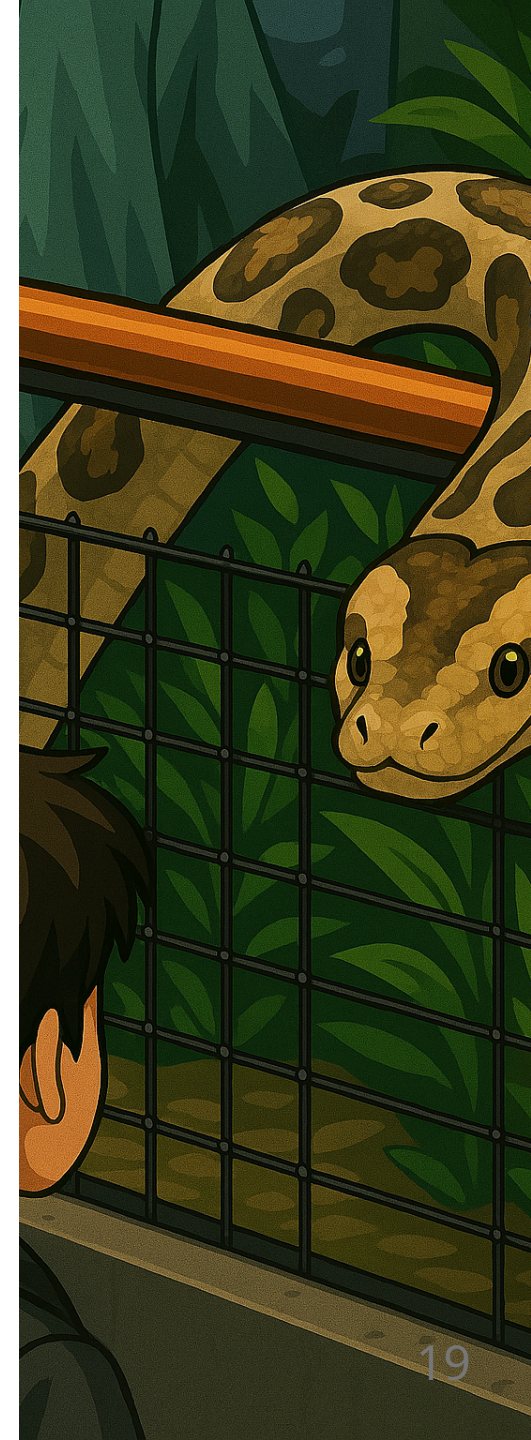
Structure	Ce que fait <code>in</code>	Complexité moyenne	Remarques
<code>set</code>	Table de hachage	O(1) (Constant)	Très rapide, pas de doublons, éléments hachables
<code>dict</code>	Table de hachage clé	O(1)	Vérifie uniquement les clés
<code>list</code>	Parcours linéaire	O(n) (Linéaire)	Compare chaque élément un à un
<code>tuple</code>	Parcours linéaire	O(n)	Comme une liste, mais immuable

 La complexité évalue le coût d'un algorithme en fonction de quantité de données.

Parcours à l'aide d'une boucle

Ces structures de données contiennent plusieurs éléments et sont dites **itérables**. Il est donc possible de les parcourir **élément par élément** à l'aide d'une boucle.

i La structure `for x in s` est particulièrement adaptée pour manipuler ce type d'objet.



Parcours à l'aide d'une boucle - liste

```
ma_liste = [5, 2, 4, 8, 9]

for i in ma_liste:
    print(i)

# Résultat : 5, 2, 4, 8, 9

ma_liste.sort()

for i in ma_liste:
    print(i)

# Résultat : 2, 4, 5, 8, 9

for i in reversed(ma_liste):
    print(i)

# Résultat : 9, 8, 5, 4, 2
```

Parcours à l'aide d'une boucle - n-uplet

```
mon_tuple = (5, 2, 4, 8, 9)

for i in mon_tuple:
    print(i)

# Résultat : 5, 2, 4, 8, 9

for i in reversed(mon_tuple):
    print(i)

# Résultat : 9, 8, 4, 2, 5
```

Parcours à l'aide d'une boucle - dictionnaire

```
mon_dict = {"a": 5, "b": 2, "c": 4}

for cle in mon_dict:
    print(cle)

# Résultat : a, b, c (dans l'ordre d'insertion)

for cle, valeur in mon_dict.items():
    print(cle, "→", valeur)

# Résultat : a → 5, b → 2, c → 4

# Parcours des valeurs uniquement
for valeur in mon_dict.values():
    print(valeur)

# Résultat : 5, 2, 4
```

Parcours à l'aide d'une boucle - ensemble

```
mon_ensemble = {5, 2, 4, 8, 9}
```

```
for i in mon_ensemble:  
    print(i)
```

```
# Résultat (ordre non garanti) : par exemple 2, 4, 5, 8, 9
```

Exercices



Liens et sources

- <https://docs.python.org/3/tutorial/datastructures.html#tuples-and-sequences>